Programmation parallèle

- OpenMP-

Ichrak MEHREZ
(ichrak.mehrez@fst.utm.tn)

Définition

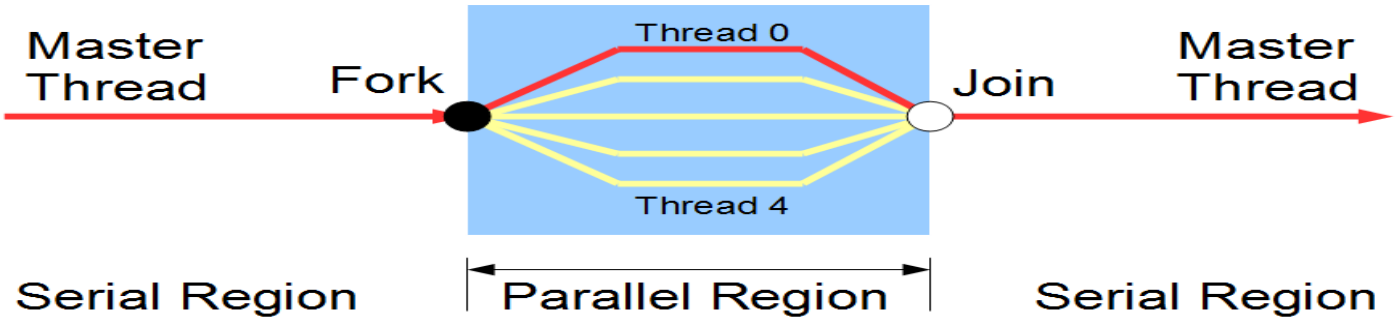
- OpenMP (Open MultiProcessing): API de programmation à base de **directives** pour l'implémentation des programmes sur des systèmes parallèles à *mémoire partagée.*

- L'approche la plus simple pour paralléliser un code séquentiel avec OpenMP consiste à paralléliser les boucles. Pour le faire, il s'agit de lancer plusieurs threads, chacun exécute une ou plusieurs itérations d'une boucle.

- OpenMP possède une interface de programmation pour les langages de programmation C, C++ et Fortran, et il est supporté par de nombreux environnements (e.g. Linux ou Windows).

Modèle d'exécution

- Modèle *fork-join* d'exécution parallèle
- L'exécution commence avec un seul processus (thread maître)
- Début d'une construction parallèle : le thread maître crée un ensemble de threads
- Fin d'une construction parallèle. Les threads de l'ensemble se synchronise : barrière implicite
- Seul le thread maître continue à s'exécuter



Avant de commencer

Pour pouvoir utiliser OpenMP, il est nécessaire de :

- inclure le fichier d'en-tête «*omp.h*» :

```
#include <omp.h>
```

- Compiler avec l'option «*-fopenmp*» :

```
$ gcc -fopenmp -o Mon_prog mon_source.c
```

- Fixer le nombre de threads :

```
$ export OMP_NUM_THREADS=2
```

Pour mesurer le temps d'exécution :

```
double temps_initial = omp_get_wtime();
 /* Travail */
 printf("Temps pris\,:%f\n", omp_get_wtime() - temps_initial);
```

Avant de commencer

- Directive #pragma omp parallel

  ➔ indiquer le début d'une région parallèle

- **Fonctions utiles**

  omp_get_num_threads ()

  ➔ le nombre de threads en cours d'exécution (Dans une région parallèle)

  omp_get_thread_num()

  ➔ le rang (numéro) du thread courant (qui fait appel à la fonction)

Un premier exemple

Soit le fameux programme « Hello World ! » suivant:

```
#include <stdio.h>
int main (int argc, char *argv[])
{ printf("Hello World !\n");
  return 0;
}
```

Ecrire une version parallèle de ce programme en utilisant OpenMP. L'affichage doit être modifié de la façon suivante : «**Hello World du thread : *<rang_du_thread_courant>*** !». Le processus du rang 0 devra afficher également le nombre total de processus crées.

Exemple 2

Soit le programme séquentiel suivant :

```
int main ()
{
      int n,i,j;
      int **a;
      n=9000;
      a=(int**) malloc (n *sizeof(int*));

      for(i=0;i<n;i++)
            {a[i]=(int*) malloc (n*sizeof(int));
             for(j=0;j<n;j++)
                  a[i][j]=i*j;
            }
      for(i=0;i<n;i++)
            free(a[i]);
      free(a);
return 0;
}
```

1) Donner la version parallèle de ce programme (pour deux threads).

2) Analyser les performances des deux programmes en comparant leurs temps d'exécution.

Thread

- Un thread: un processus léger

Un programme en cours d'exécution
Chaque processus possède sa propre mémoire virtuelle.

- Un thread est, en quelque sorte « *un processus dans un processus* »

- Les threads d'un même processus se partagent la mémoire virtuelle

A savoir

- Par défaut: nombre de threads = nbr CPUs
- Pour avoir nbr CPUs à partir du terminal

```
$ cat /proc/cpuinfo |grep processor |wc -l
```

```
int nb=sysconf(_SC_NPROCESSORS_ONLN)
```

- Avoir numéro du CPU à partir d'un programme C

```
int nb= sched_getcpu()   // ajouter #include <sched.h>
```

Variables

- **Shared :** par défaut toutes les variables sont partagées, sauf les compteurs des itérations.
- **private :** variable privée pour chaque thread, non initialisée.
- **firstprivate :** variable privée pour chaque thread, initialisée à la valeur de la variable originale.
- **lastprivate :** variable privée pour chaque thread, variable originale mise à jour après modification.
- **threadprivate (i):** variable privée pour chaque thread dont la valeur persiste d'une région // à une autre

Les clauses

- L'ensemble de threads n'est créé que lorsque expression !=0 (Si (exprerssion==0, la partie parallèle est exécutée en séquentielle)

```
#pragma omp parallel if (expression)
```

- Définir nombre de thread d'une région parallèle

```
#pragma omp parallel num_threads (nb_th)
```

- Définir si les variables dans une région parallèle sont partagées ou non

```
#pragma omp parallel default (shared|none)
```

none: force le programmeur à déclarer chaque variable dans la région parallèle

Les clauses

- Initialiser une variable de type *threadprivate* avec la valeur de la variable du thread maître

```
#pragma omp parallel  copyin(i)
```

- **copyprivate (*list_var*)** : copie les valeurs des variables privées d'un thread, modifiées dans une région *single*, dans les variables correspondantes des autres threads.

  ➔ ne peut pas être utilisé avec *nowait*

- **nowait**: annuler les barrières implicites

Directives master et single

- Région exécutée par un seul thread, le thread maître (numéro 0). Les autres threads continuent leurs traitements.

```
#pragma omp master
```

- Région exécutée par un seul thread, (le premier qui arrive, on ne peut pas le prévoir). Les autres threads sont bloqués.

```
#pragma omp single
```

- Synchronisation de threads

```
#pragma omp barrier
```

- Spécifier le nombre de thread dans une région parallèle où on n'a pas utilisé l'option *num_threads()*

```
omp_set_num_threads (nb)
```

- Vérifier si on est dans une région parallèle

```
int par = omp_in_parallel()
```

➔ Retourne TRUE (1) si région parallèle, sinon FALSE (0)

- Partie exécutée par un seul thread à la fois

```
#pragma omp critical
```

- Modification d'une variable: ++; -- ; opération logique

➔ S'assurer qu'une variable est mise à jour par un seul thread à la fois

```
#pragma omp atomic
```

Boucle For

- Parallélisation de la boucle for

```
#pragma omp parallel for
  for(i=0;i<N;i++)
```

```
#pragma omp parallel
 { .
       #pragma omp for
         for(i=0;i<N;i++)
   .
 }
```

- La variable *i* est *privée* par défaut

Boucle For: Reduction

- Parallélisation de la boucle for

```
#pragma omp parallel for reduction (op : res)
  for(i=0;i<N;i++)
```

- Une copie locale de *res* est associée à chaque thread. Chaque copie est initialisé selon l'opération ***op*** (0 dans le cas de l'addition '+')

Boucle For: Ordered

- Le code est exécuté dans un ordre séquentiel
- La directive #pragma omp ordered doit être appelé seulement dans #pragma omp parallel for ordered ou #pragma omp for ordered

```
#pragma omp parallel for ordered
 for(i=0;i<N;i++)
{  ...
   …
     #pragma omp ordered
       printf(« i=%d \n »,i);
   …
   …
}
```

Boucle For: Schedule (1/5)

- Permet de définir la distribution des itérations entre les threads

```
#pragma omp parallel for schedule (type, taille_bloc)
  for(i=0;i<N;i++)
```

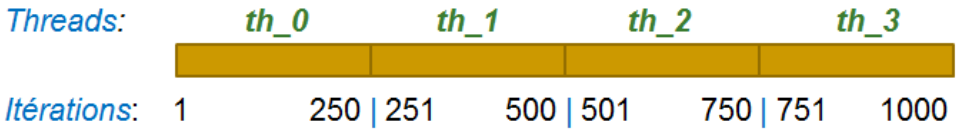
- type ∈ {*static*, *dynamic*, *guided*, *runtime*, *auto*}
- *taille_bloc* : nombre d'itérations pour chaque thread

Boucle For: Schedule (2/5)

- type = *static*

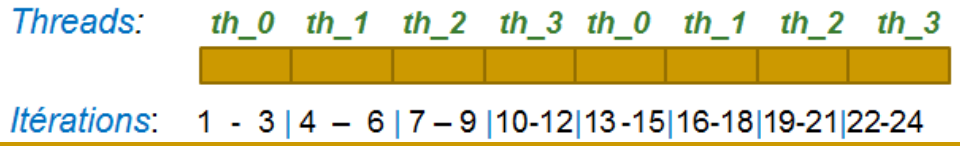
```
#pragma omp parallel for schedule (static, taille_bloc)
  for(i=0;i<N;i++)
```

- Correspond à la répartition par défaut: N/p itérations affectées à chaque thread, avec p correspond au nombre total des threads



- Exemple:

```
#pragma omp parallel for schedule (static, 3)
  for(i=0;i<N;i++)
```



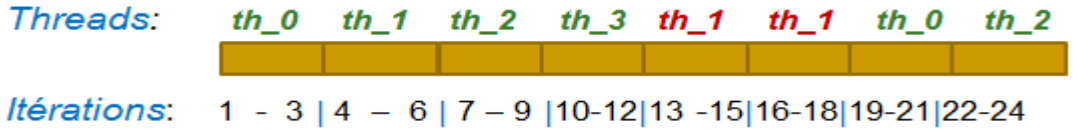
Boucle For: Schedule (3/5)

- type = *dynamic*

```
#pragma omp parallel for schedule (dynamic, taille_bloc)
  for(i=0;i<N;i++)
```

- Quand un thread termine son bloc d'itérations, il passe au bloc suivant (le premier dans la liste d'attente).
- Utilisé généralement lorsque la taille des itérations n'est pas le même.
- Exemple:

```
#pragma omp parallel for schedule (dynamic, 3)
  for(i=0;i<N;i++)
```

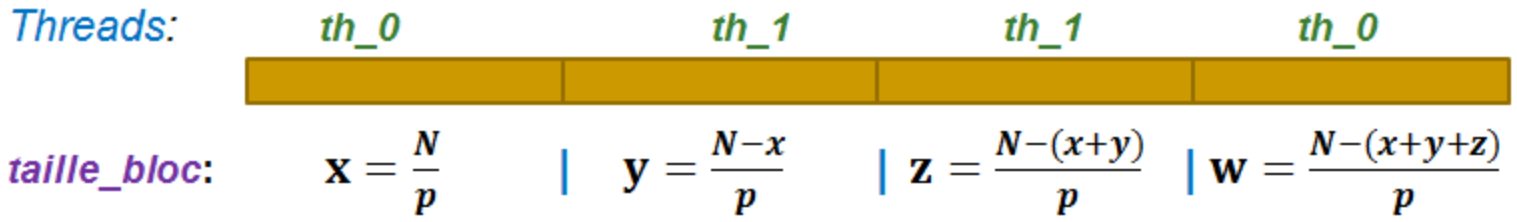


Boucle For: Schedule (4/5)

- type = *guided*

```
#pragma omp parallel for schedule (guided, taille_bloc)
  for(i=0;i<N;i++)
```

- Similaire à ***dynamic*** sauf que la taille du bloc suivant dépends du nombre des itérations qui restent



Boucle For: Schedule (5/5)

- type = *runtime*

```
#pragma omp parallel for schedule (runtime)
  for(i=0;i<N;i++)
```

➔ le choix est précisé lors de l'exécution

- type = *auto*

```
#pragma omp parallel for schedule (auto)
  for(i=0;i<N;i++)
```

➔ le choix est laissé au compilateur

Section

- Partage de travail: les sections sont exécutées en parallèle. Chaque section est exécutée par un thread différent.

```
#pragma omp sections
{
        #pragma omp section
        {… }

        #pragma omp section
        {… }

}
```